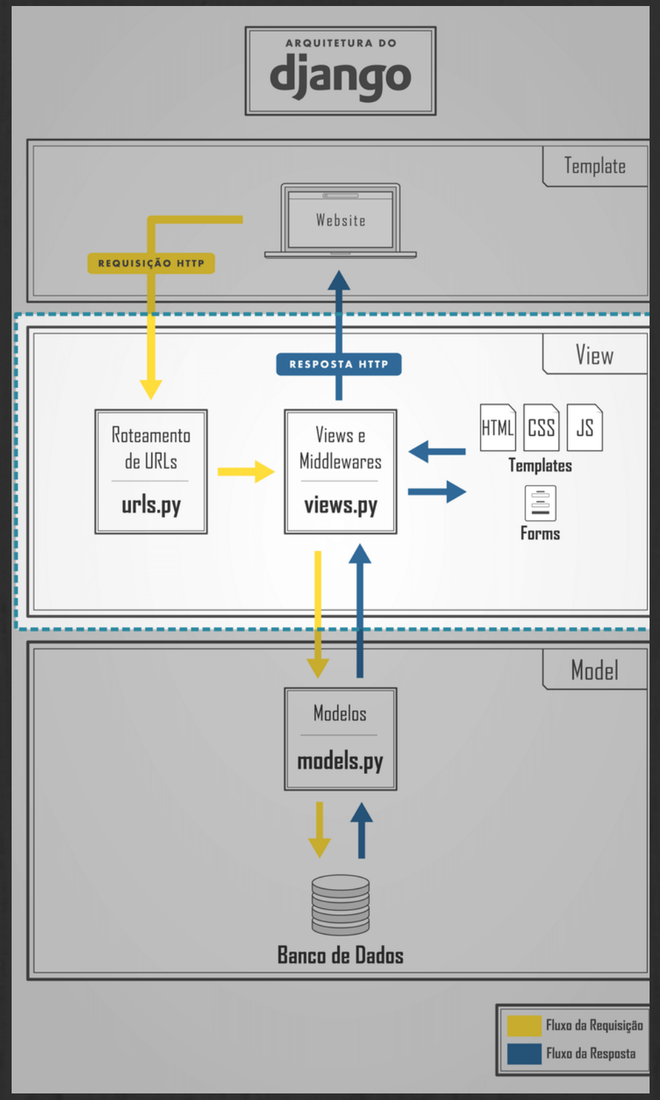
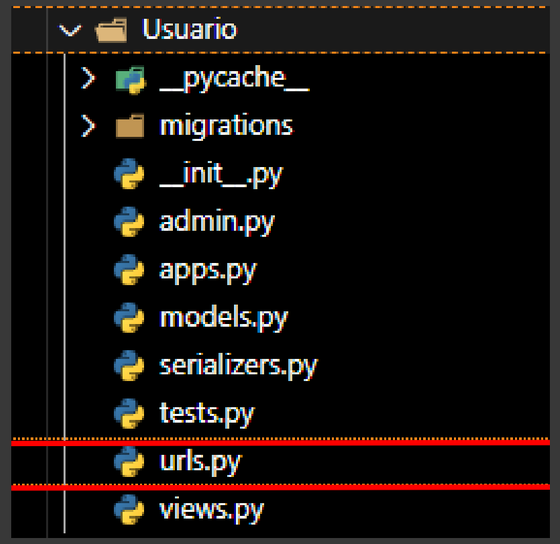
Iniciando a parte lógica



No Django, as URLs são mapeadas para funções de visualização em um arquivo urls.py. Quando um usuário acessa uma URL específica, a função da view correspondente é chamada. As visualizações processam a lógica da aplicação, que pode incluir interações com o banco de dados. Elas utilizam modelos para criar páginas HTML dinâmicas, que são enviadas de volta ao navegador do usuário como resposta à solicitação original. Esse processo facilita a criação de aplicativos da web robustos e dinâmicos.

Configurando as urls do APP

Dentro do seu APP crie um arquivo urls.py



ATENÇÃO-> **Não confunda as urls que estamos criando agora, com as que criamos no PDF anterior essas urls são especificas do app, já a anterior era da base do projeto.**

Dentro de urls.py

```python
from django.urls import path  # Importa a função 'path' do módulo 'django.urls'
from . import views  # Importa as visualizações definidas no arquivo atual
from django.conf import settings  # Importa as configurações do Django
from django.conf.urls.static import static  # Importa a função 'static' para servir arquivos estáticos durante o desenvolviment

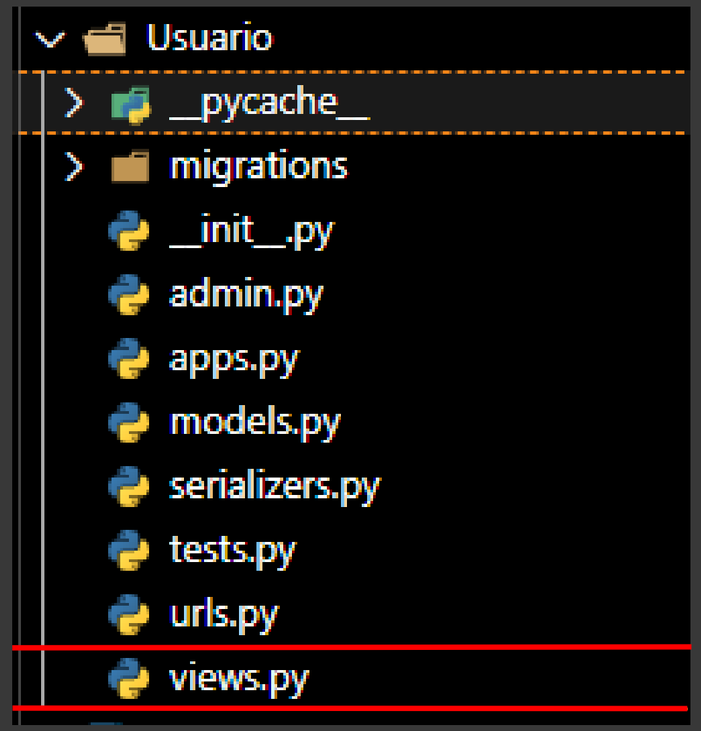
urlpatterns = [

]

if settings.DEBUG:
    urlpatterns+= static(settings.MEDIA_URL,document_root=settings.Media_root) # para acessar midias via url.
```

Dentro do url patterns é onde iremos posicionar todas as urls necessárias para a aplicação

Acesse o arquivo views.py



Imports views.py

```python
from django.shortcuts import render, redirect, get_object_or_404  # Importa funções para renderizar templates, redirecionar e obter objetos do banco de dados ou gerar erro 404
from .models import Usuario  # Importa o modelo 'Usuario' definido no arquivo atual
from django.urls import reverse  # Importa a função 'reverse' para construir URLs reversas
from django.core import serializers  # Importa um mecanismo para serializar/deserializar dados do Django
from .serializers import UsuarioSerializer # Importa os serializadores definidos no arquivo atual
from rest_framework.renderers import TemplateHTMLRenderer  # Importa o renderizador de templates do Django REST Framework
from rest_framework.response import Response  # Importa a classe de resposta do Django REST Framework
from rest_framework import status, generics  # Importa códigos de status HTTP e classes genéricas do Django REST Framework
```

A view é onde ficará toda a lógica do nosso APP e a maioria das funções que criamos precisam de importes específicos

Cadastrando Usuário

Agora que fizemos tudo que era necessário, iremos criar a primeira funcionalidade

```
urlpatterns = [
    path('',views.CadastrarUsuario.as_view(),name='cadastro')

]
```

Primeira URL que criamos é a de cadastrar usuário,
usamos o name para acessar a URL dentro do html

Criando a View de cadastro

```python
class CadastrarUsuario(generics.CreateAPIView):
    # Define o serializador a ser utilizado para criar um novo usuário
    serializer_class = UsuarioSerializer

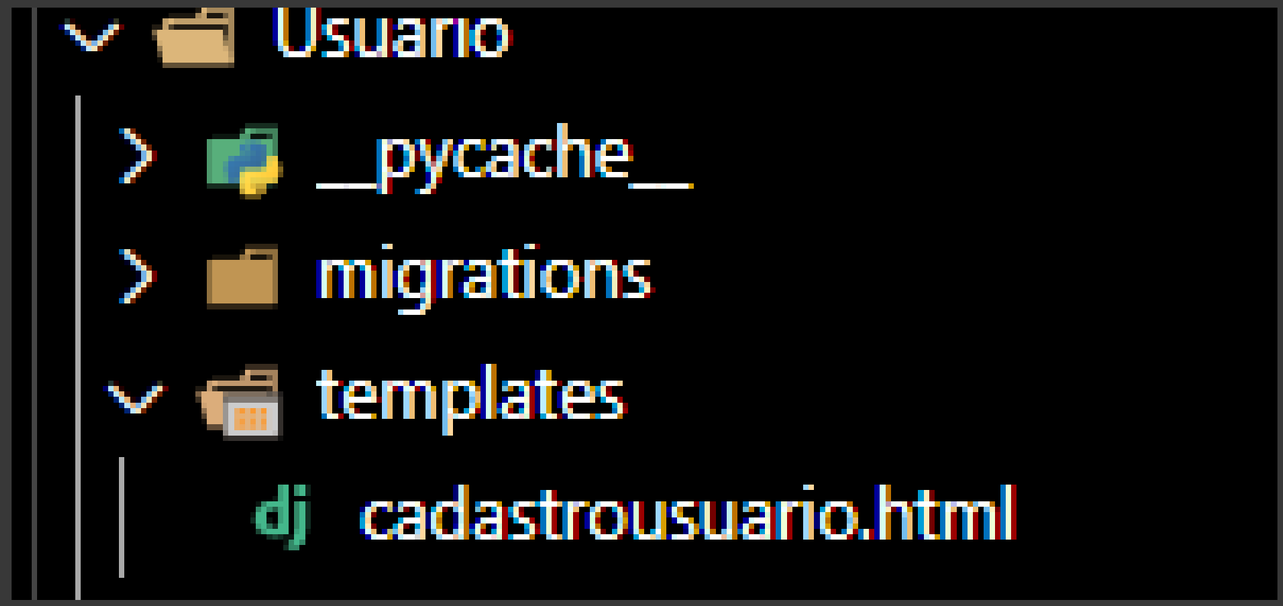
    # Método GET para exibir o formulário de cadastro do usuário
    def get(self, request, *args, **kwargs):
        # Inicializa o serializador vazio
        serializer = self.serializer_class()
        # Renderiza o template 'cadastrousuario.html', passando o serializador como contexto
        return render(request, 'cadastrousuario.html', {'serializer': serializer})

    # Método POST para processar o formulário de cadastro do usuário
    def post(self, request, *args, **kwargs):
        # Inicializa o serializador com os dados da requisição
        serializer = self.serializer_class(data=request.data)

        # Verifica se os dados do formulário são válidos
        if serializer.is_valid():
            # Salva o novo usuário no banco de dados
            serializer.save()
            # Redireciona para a página de cadastro bem-sucedida
            return redirect('cadastro')
```

Usamos o generics.CreateAPIView para criar a função de cadastro, tratamos o método GET e POST do navegador.

Inserindo o html



Dentro da pasta do nosso APPcriaremos outra pasta templates, onde nela criaremos nosso HTML DE para a tela de cadastro de usuário, o nome deve ser exatamente igual, a como foi passado na view na pagina passada.

(HTML disponível no repositório)

Entendendo html no django

```
{% load static %}  <!-- Carrega os arquivos estáticos no template -->
<html lang="pt-BR">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Projeto Django{% endblock %}</title>  <!-- Define o título da página com um bloco que pode ser substituído em templates filhos -->
    {% block head %}  <!-- Inicia um bloco chamado 'head' que pode ser substituído em templates filhos -->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">  <!-- Inclui o arquivo CSS da biblioteca Bootstrap -->
    <link rel="stylesheet" href="{% static 'general/base.css' %}">  <!-- Inclui um arquivo CSS estático chamado 'base.css' do diretório 'general' dos arquivos estáticos -->
    {% endblock %}  <!-- Fecha o bloco 'head' -->
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-light bg-light">
        <h1 class="titulo">Cadastrando usuario</h1>  <!-- Define o título da página na barra de navegação -->
        <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="fals
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarSupportedContent">
            <ul class="navbar-nav mr-auto">
                <li class="nav-item active">
                    <a class="nav-link" href="#">Cadastrar usuario <span class="sr-only"></span></a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="#">Gerenciar usuarios</a>
                </li>
            </ul>
        </div>
    </nav>

    <div class="container mt-5">
        {% block dashboard %}  <!-- Inicia um bloco chamado 'dashboard' que pode ser substituído em templates filhos -->
    <!-- onde ficará o conteudo da nossa pagina -->
        {% endblock %}  <!-- Fecha o bloco 'dashboard' -->
    </div>
```

Todos os htmls que iremos criar, irão estender da base, a base geralmente possui algo que se repetirá durante toda a aplicação, como uma nav-bar e o fundo por exemplo, e para não precisarmos repetir código, estendemos essa base.

Entendendo html no django

```
{% extends 'base.html' %}  <!-- Estende o template 'base.html' -->
{% load static %}  <!-- Carrega os arquivos estáticos no template -->
{% load rest_framework %}  <!-- Carrega as tags e filtros do Django REST Framework -->

{% block dashboard %}  <!-- Inicia um bloco chamado 'dashboard' que pode ser substituído em templates filhos -->
    <h2>Cadastro de usuario </h2>  <!-- Título da página de cadastro de usuários -->
    <form method="post" enctype="multipart/form-data" class='form-control'>
        {% csrf_token %}  <!-- Token de segurança CSRF para proteger contra ataques CSRF -->
        {% render_form serializer %}  <!-- Renderiza o formulário usando o serializer passado como contexto -->
        <br>
        <button type="submit" class="btn btn-primary">Cadastrar usuario</button>
    </form>
{% endblock %}  <!-- Fecha o bloco 'dashboard' -->
```

Reparem no HTML de cadastro de usuário, como a primeira coisa que fazemos é estender de base.html, esses comandos são colocados nas TAGS do DJANGO representadas por {%%}
Podemos usar estruturas de decisão e de repetição nessas TAGS.